

Herramienta auxiliar de detección de plagio

Integrantes del proyecto:

- **Diego Reséndiz Fernández** A01708017
- **Félix Javier Rojas Gallardo** A01201946
- **Jesús Ramírez Delgado** A01274723

| 1.0 Introducción

En los últimos años se ha experimentado un crecimiento en el uso de la IA, lo que ha abierto un mundo de posibilidades pero también retos significativos, siendo uno de los más grandes el **plagio de código por IA**.

1.1 Objetivos

El objetivo primordial de este proyecto es **detectar el plagio en código de Python** mediante el desarrollo de un modelo MLP (Multi-Layer Perceptron) especializado.



| 2.0 Marco Teórico

Fundamentos de Detección de Plagio e IA

El plagio se define como la detección de duplicidad estructural o semántica. Los modelos modernos utilizan **Deep Learning** y **NLP** para superar el simple matching de texto, analizando relaciones lógicas complejas.

Arquitectura Híbrida

Propuesta de Suh et al.: Combinar métricas estáticas con **AST (Abstract Syntax Tree)** para capturar cómo está escrito y organizado el código.

Índice de Tolerancia

Uso de rangos (0.0 - 1.0) para clasificar entre Humano, Duda e IA, permitiendo un margen de error controlado y flexible.

Tipos de Plagio y Técnicas de Detección

Verbatim: Copia tal cual.

Paráfrasis: Cambios con misma lógica.

Traducción: Cambio de lenguaje.

Métricas clave: Conteo de estructuras de control (if/else), bucles, profundidad del AST, conteo de funciones, radio de comentarios, número de variables, Distancia Levenshtein y complejidad ciclomática.



| 3.0 Dataset y Corpus de Entrenamiento

Origen y Composición

El dataset base proviene de repositorios académicos de GitHub (Ehsankhani) con códigos en Python. Para resolver el desbalance inicial, se expandió el corpus mediante generación sintética con LLMs.

Fuentes Generativas

- Deepseek v4 flash
- Gemini v2.5 flash
- Mistral Medium 3.1

Estrategia de Split

80% Entrenamiento / 20% Prueba

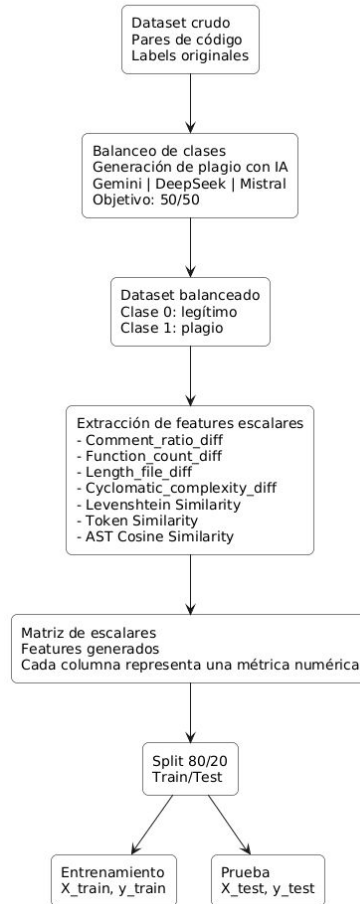
Tipología de Muestras

- Código Humano
- Código IA
- Humano → IA (Ofuscado)
- IA → IA (Parafraseado)

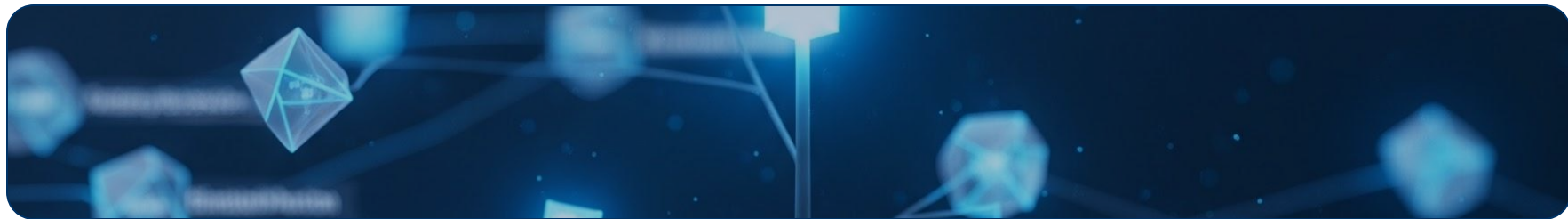
Objetivo: Lograr una generalización robusta ante múltiples modelos.



Pipeline ETL del Dataset



| 4.0 Selección y Justificación de Features



Similitud Léxica

Token Similarity

Compara lógica de símbolos ignorando espacios; usa Jaccard.

Levenshtein

Detecta copia directa midiendo la distancia de edición textual.

Estructural (AST)

TF-IDF AST

Captura el estilo de escritura mediante relaciones de tokens en tríadas, mitigando la ofuscación por renombramiento.

MRT OAST

Analiza la estructura del código del árbol sintáctico recorrido pre-order y post-order. Usamos el modelo GraphCodeBERT para destilar la similitud encontrada a un escalar.

Atributos Relativos

Comment Ratio: Delata ofuscación.

Function Count: Sospecha de copia.

File Length: Indicador de duplicidad.

Variable count: Indicador de duplicidad.

Complejidad Ciclomática: Filtro de estructura.

La red neuronal identifica la distancia lógica y estructural entre pares de códigos mediante diferencias normalizadas.

| Feature: Levenshtein Similarity

Detección de Copia Directa

- **Base de Cálculo:** Se basa en la "distancia de edición," que mide cuántos cambios de un solo carácter son necesarios para transformar un código en otro.
- **Propósito:** Es eficaz para detectar el plagio por "copia y pega" directo con modificaciones mínimas, ya que la distancia de edición textual es mínima.
- **Aporte al Modelo:** Actúa como un detector de duplicidad textual cruda, identificando similitudes superficiales que métricas más complejas podrían ignorar por su enfoque estructural.

Source String

LEVENSTINES

Target String

LEVENSHTEIN'S

Edits Needed

LEVENSHTEIN'S
~~E~~

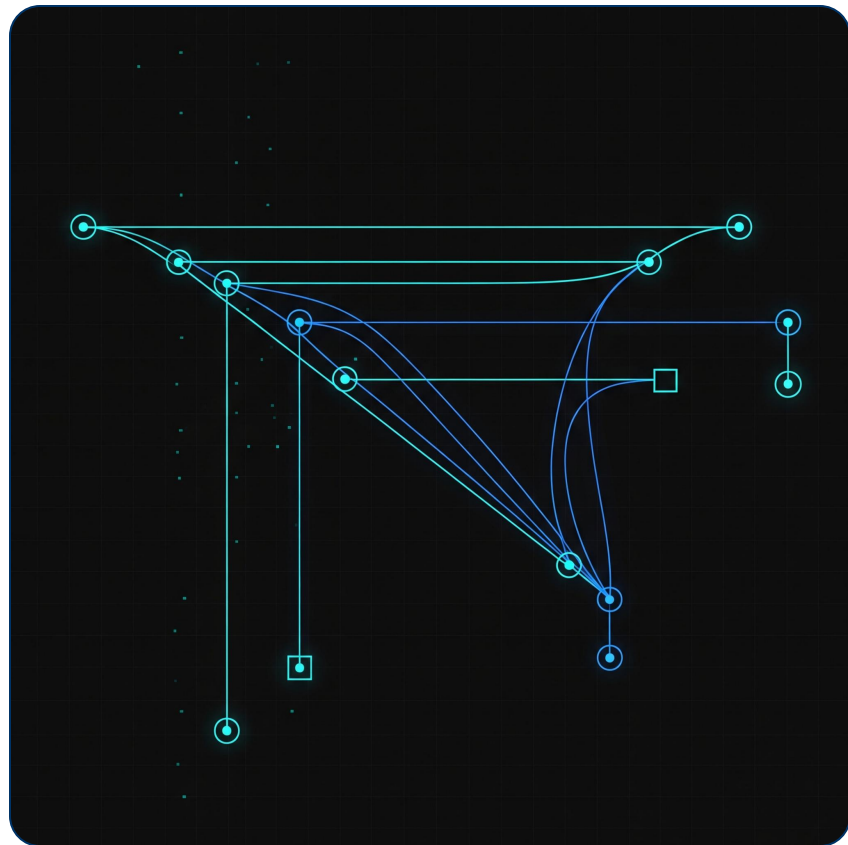
Levenshtein's Distance

4 (3 additions + 1 deletion)

| Feature: Complejidad Ciclomática

Huella de Estructura Lógica

- **Base de Cálculo:** Utiliza **radon.complexity** para sumar caminos lógicos (*if*, *while*, *for*). El modelo analiza la diferencia normalizada entre pares de archivos para identificar duplicidad.
- **Propósito:** Funciona como un filtro de alta fidelidad contra el plagio. Es una métrica "difícil de ocultar" ya que la complejidad estructural suele preservarse incluso tras renombramientos superficiales.
- **Aporte al Modelo:** Crucial para mitigar falsos positivos al confirmar que la lógica funcional subyacente es idéntica, reforzando la precisión de la red neuronal.



| Features: Atributos diferenciales relativos

Detección de Ofuscación parcial

Base de Cálculo: La función calcula la proporción de líneas con comentarios (#) respecto al total de líneas del archivo, conteo de funciones, variables y longitud de los archivos.

Propósito: Identificar anomalías en la documentación del código. Una diferencia mínima entre archivos es altamente sospechosa.

Aporte al Modelo: Delata técnicas de ofuscación donde el plagiador mantiene la misma densidad de explicaciones o las elimina.

Comment Ratio

Delata ofuscación

Function Count

Sospecha de copia

File Length

Indicador de duplicidad

Variable Count

Indicador de duplicidad



| Feature: Token Similarity

Análisis Léxico Profundo

- **Base de Cálculo:** El código se transforma en una secuencia de tokens (identificadores, palabras clave y operadores) eliminando comentarios y espacios en blanco. Se hacen conjuntos y se calcula la **Distancia de Jaccard**.
- **Propósito:** Evalúa la consistencia de la lógica simbólica. Ayuda a detectar plagios donde solo se alteraron nombres de variables o el formato de manera superficial.
- **Aporte al Modelo:** Ofrece una capa de seguridad contra técnicas de ofuscación comunes, identificando la estructura funcional subyacente del programa.

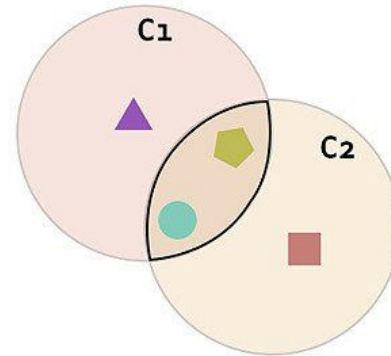


Jaccard Similarity

COMPUTE THE SIMILARITY BETWEEN TWO OBJECTS

	●	■	▲	◆
C1	●		▲	◆
C2	●	■	◆	◆
C3	●		▲	

$\text{sim}(c1, c2) \rightarrow$



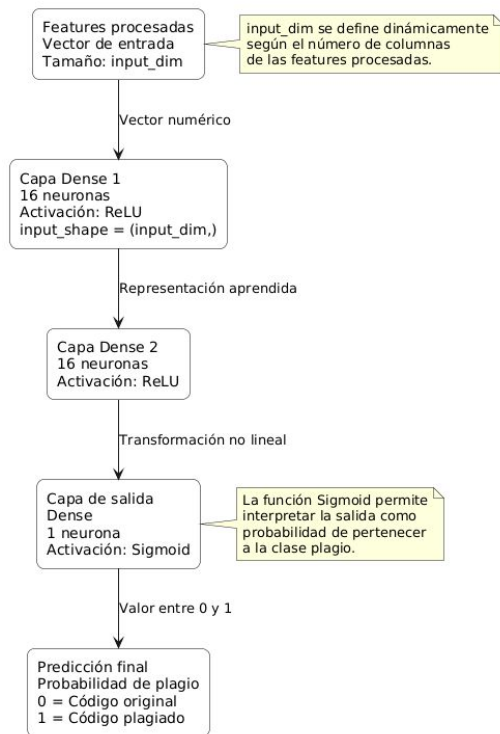
JACCARD SIMILARITY

$$\frac{\text{green circle} + \text{green pentagon}}{\text{green circle} + \text{green pentagon} + \text{red square} + \text{purple triangle}} = \frac{2}{2 + 1 + 1} =$$

0.5



| Arquitectura: MLP Version 1



| Arquitectura: MLP Version 2

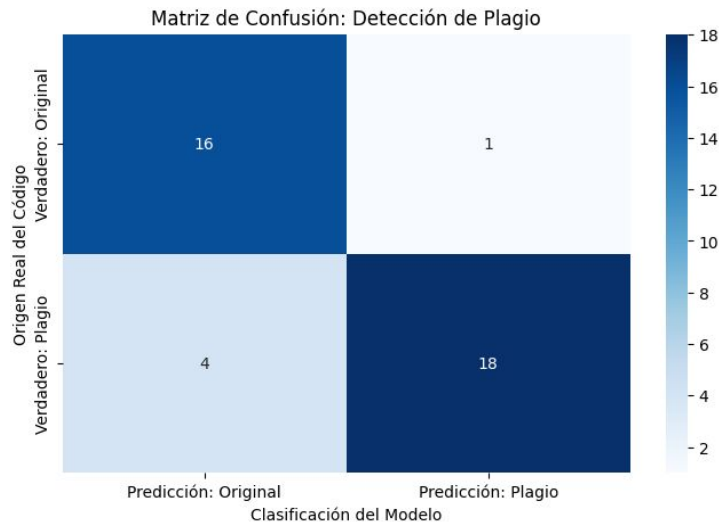
Evolución hacia un Modelo Conservador

La versión 2 mantiene la **integridad arquitectónica** del MLP original, centrando su optimización en la sensibilidad de clasificación.

- **Consistencia Estructural:** No se realizaron cambios en las capas densas ni en las funciones de activación, asegurando estabilidad funcional.
- **Ajuste de Umbral (Threshold):** Se incrementó la exigencia del modelo para la detección de plagio.
- **Objetivo:** Reducir falsos positivos mediante un criterio de clasificación más "conservativo" y riguroso.

Threshold: 0.5 a 0.8

de Parámetros

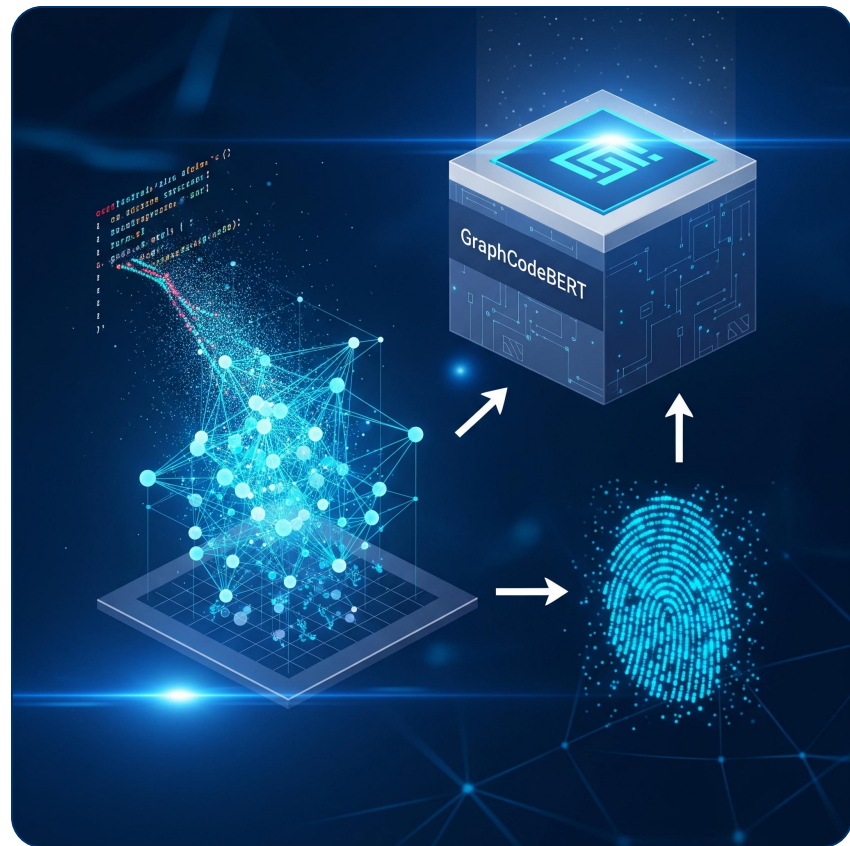


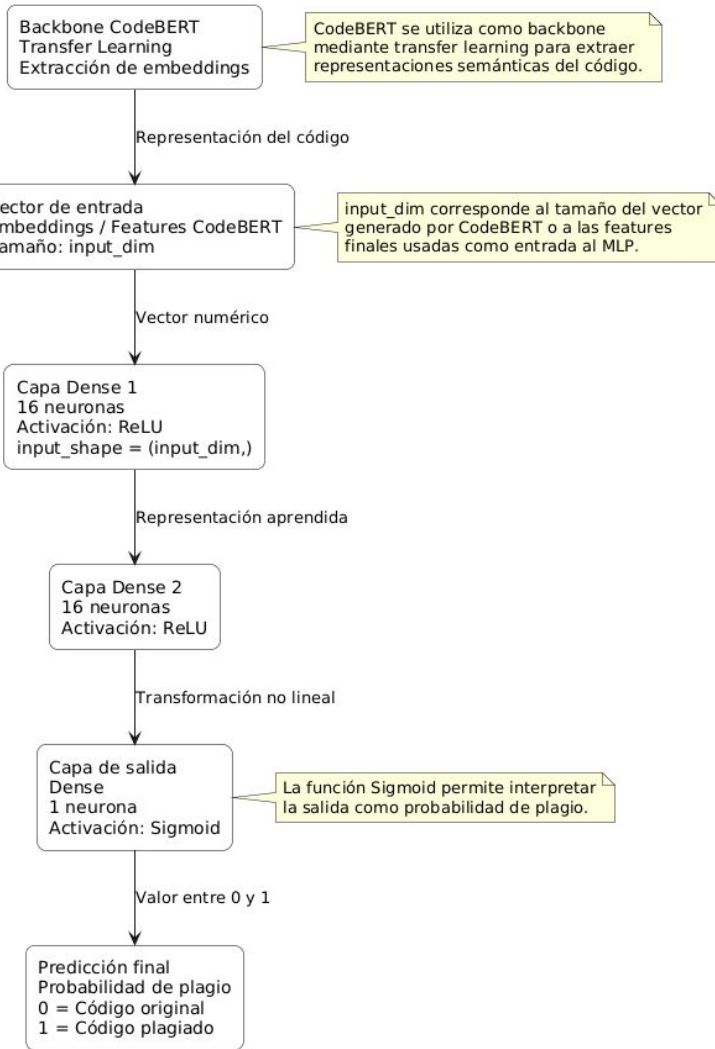
Arquitectura: MLP Version 3 Integración de GraphCodeBERT

Análisis Semántico Profundo

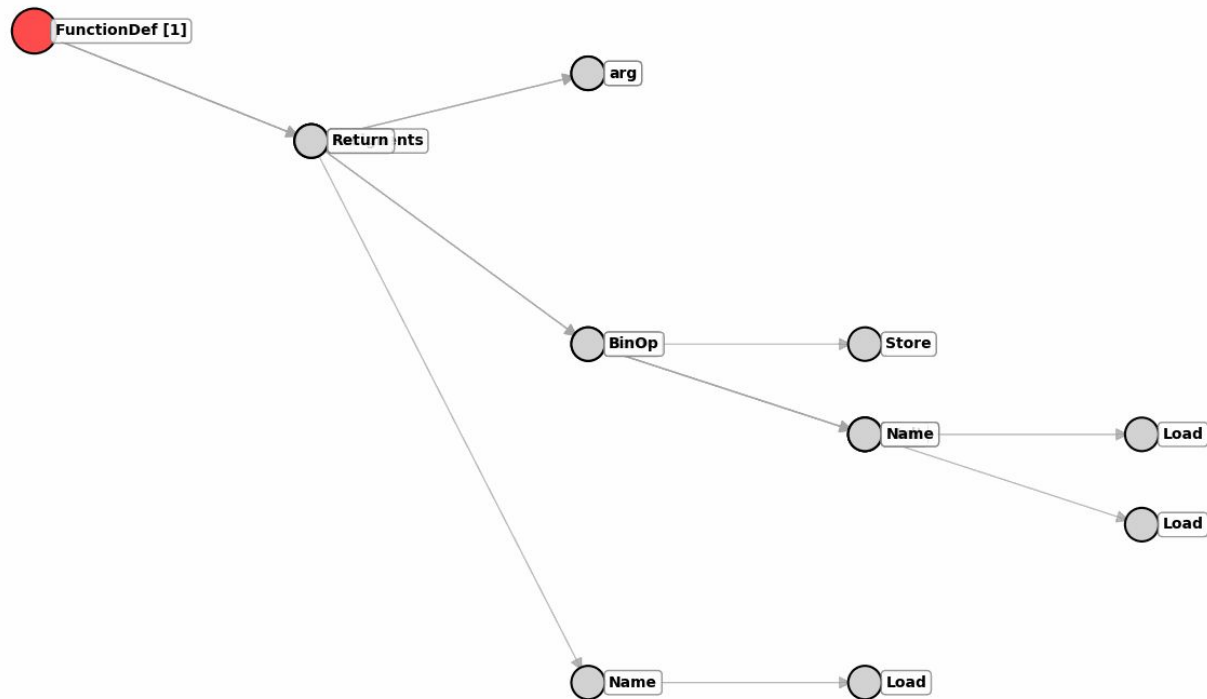
El sistema utiliza **GraphCodeBERT** para transformar la lógica del código en una firma digital semántica.

- **OAST (Optimized AST):** Filtra librerías e importaciones para centrarse exclusivamente en la lógica central del código.
- **Recorridos Estructurales:** Genera secuencias mediante Pre-order y Post-order para capturar jerarquías y relaciones entre terminales.
- **Embeddings:** Genera vectores de 768 dimensiones que actúan como una "huella digital", permitiendo detectar plagio tras procesos de ofuscación.
- **Métrica mrt_oast:** Traduce la salida de CodeBERT en un coeficiente de similitud coseno para el MLP final.



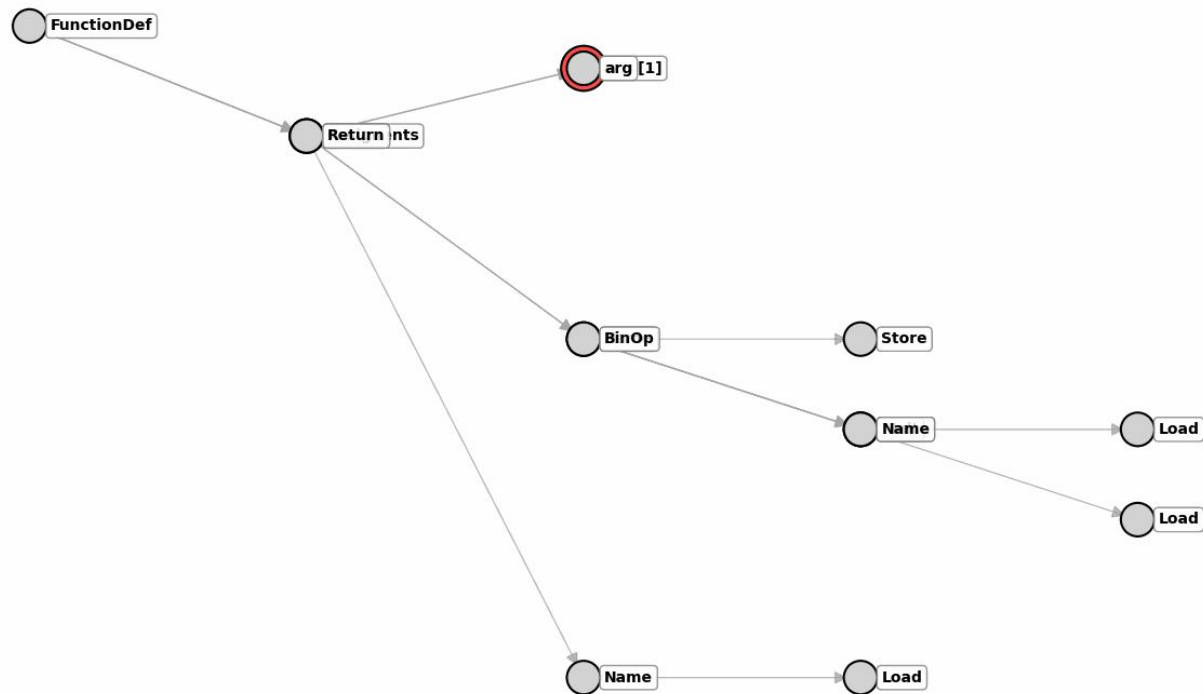


Pre-order Traversal | Step 1: FunctionDef



Array: [FunctionDef]

Post-order Traversal | Step 1: arg



Array: [arg]

| Resultados: Evaluación del Desempeño

Métricas por Modelo

Modelo	Recall	Precisión	F1-Score
Regresión Logística	0.84	0.67	0.74
MLP Versión 1	0.82	0.95	0.88
MLP Versión 2	0.59	0.93	0.72
MLP Versión 3	0.73	0.94	0.82

La **Precisión** es nuestra métrica primaria para asegurar la veracidad del diagnóstico y evitar falsas alarmas.

La **Versión 3** (GraphCodeBERT) logra el mejor equilibrio frente a la inflexibilidad del umbral 0.8 en la V2, la cual provocó una disminución general en todas las métricas.



Análisis de Resultados

- Integración de CodeBERT mejora la detección de **paráfrasis semántica**.E implica una mejora de 1 punto entero en todo.
- Alta robustez ante procesos de **ofuscación** compleja.

| Desventajas y Limitaciones del Modelo

Restricciones Técnicas y Sesgos

- **Herramienta Auxiliar:** El modelo está concebido estrictamente como un soporte técnico para supervisión, no como un juicio final.
- **Comparación de Archivos Únicos:** No soporta la evaluación de proyectos cuya lógica se encuentre distribuida en múltiples archivos.
- **Sesgo por Sencillez:** En códigos muy cortos, existe un sesgo que puede marcar erróneamente el código como plagio debido a la naturaleza dinámica de Python.
- **Alcance de Taxonomía:** Limitado a niveles L1, L2 y L3 (cambios de variables y flujo). No detecta plagio de nivel L4 a L6 (cambios de paradigma o implementación total).



Ventajas del Modelo

Optimización y Versatilidad

- **Eficiencia en Procesamiento:** Rápida iteración sobre los datasets de entrenamiento.
- **Arquitectura Ligera:** Consumo reducido de memoria RAM gracias a su estructura optimizada.
- **Ajuste Paramétrico (Fine-tuning):** Alta capacidad para calibrar umbrales de decisión según necesidades institucionales.
- **Especialización en Python:** Enfoque único en un lenguaje dinámico, superando limitaciones de herramientas mas generales.

